Object Oriented Programming(C++)
(CSE 1211)

Polymorphism

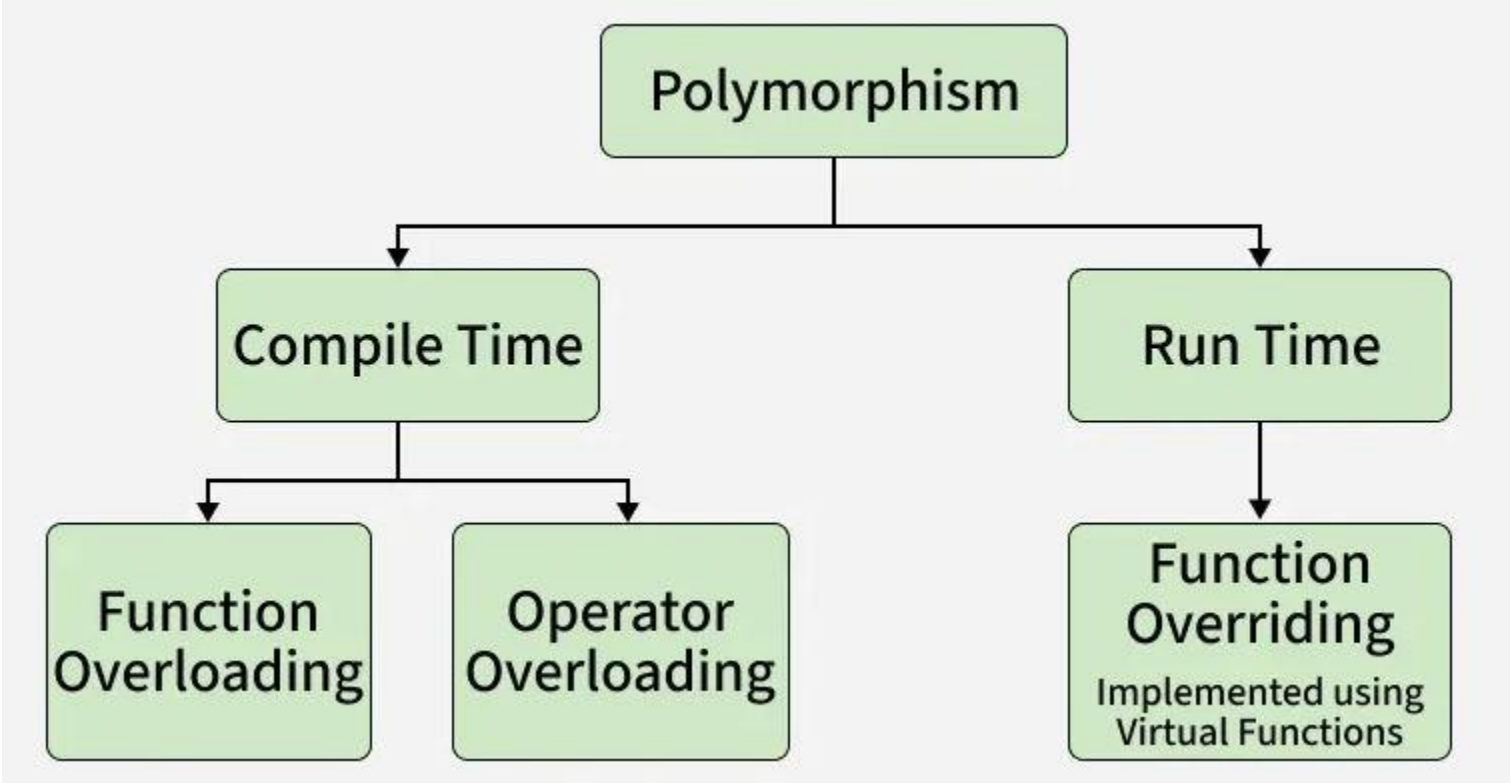
Md. Mahfuzur Rahman
Lecturer,
Prime University

Polymorphism

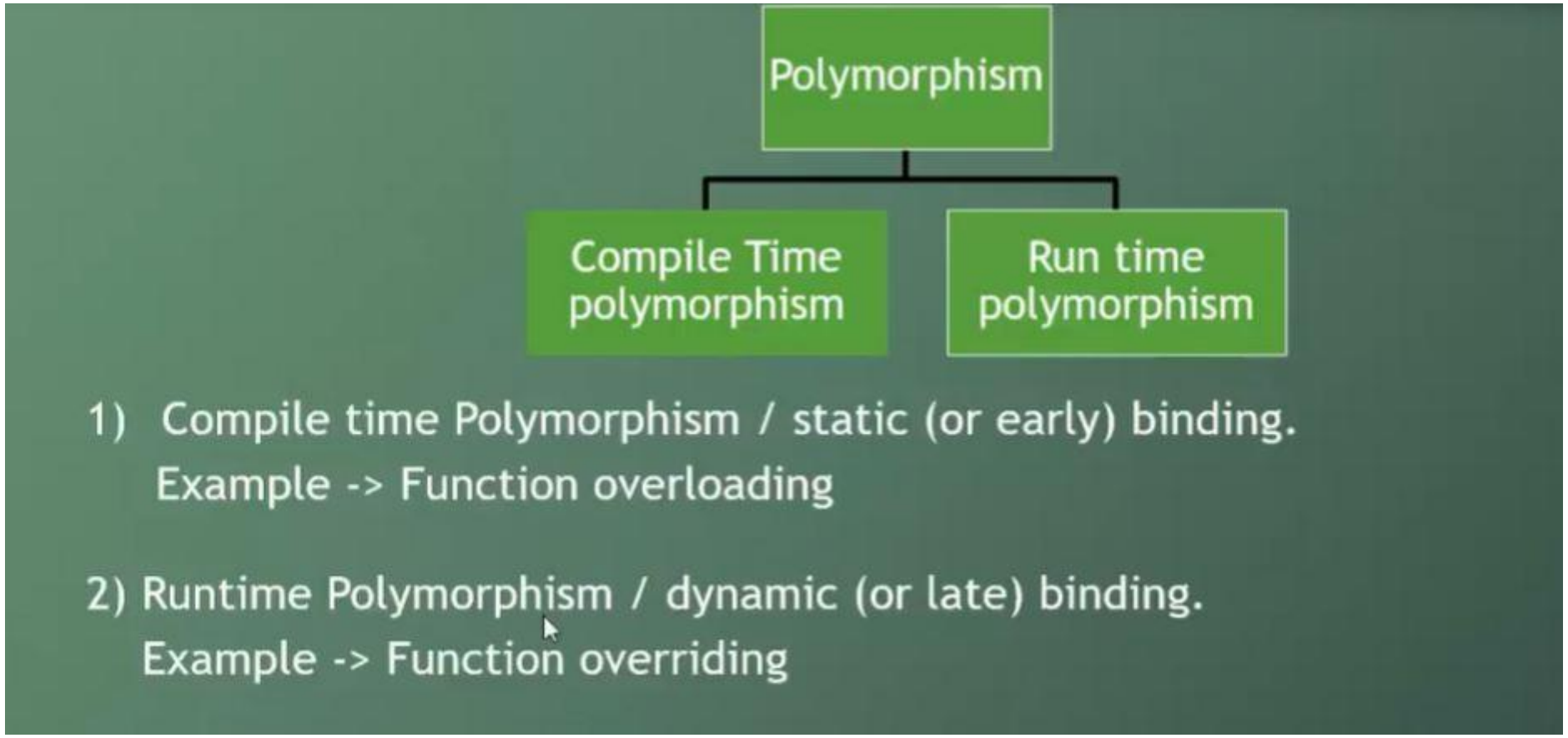
- The word **polymorphism** means having many forms. In C++, polymorphism concept can be applied to functions and operators. A single function name can work differently in different situations. Similarly, an operator works different when used in different context.
- **Polymorphism is a mechanism where parents class’s references variable refer sub class.**
- Poly means **many** and morph means **forms** or shape. Thus the word Polymorphism comes from Greek and it basically means having many forms.
- **For example,** Your DAD is your father. He's your mom's Husband. He's your grandma's son as well. Thus here your DAD is exhibiting many forms. He's a father, son, husband, brother, etc.

Polymorphism

Polymorphism means many forms. It is the ability to take more than one form. It is a feature that provides a function or an operator with more than one definition. It can be implemented using function overloading, operator overload, function overriding, and virtual functions.



Polymorphism



1. Compile-Time Polymorphism

Also known as **early binding** and **static polymorphism**, in compile-time polymorphism, the compiler determines how the function or operator will work depending on the context. This type of polymorphism is achieved by function overloading or operator overloading.

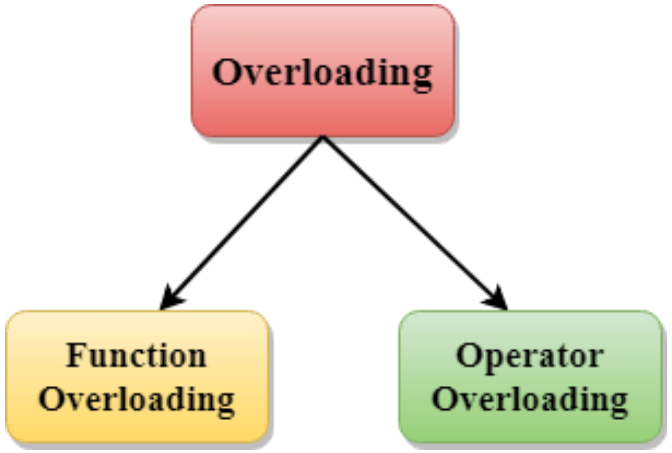
In compile-time polymorphism, the compiler determines which function or operation to call based on the number, types, and order of arguments. It is also called Static Polymorphism as the function calls are statically binded to its definition.

It is further categorized into two types:

- Function Overloading
- Operator Overloading

C++ Overloading (Function and Operator)

Types of overloading in C++ are:



C++ Function Overloading

- Function Overloading is defined as the process of having two or more function with the same name, but different in parameters is known as function overloading in C++. In function overloading, the function is redefined by using either different types of arguments or a different number of arguments. It is only through these differences compiler can differentiate between the functions.
- The **advantage** of Function overloading is that it increases the readability of the program because you don't need to use different names for the same action.

A. Function Overloading

- Function overloading is a feature of object-oriented programming where **two or more functions can have the same name** but behave differently for different parameters. Such functions are said to be **overloaded;** hence, this is known as **Function Overloading** Functions can be overloaded either by **changing the number of arguments** or **changing the type of arguments**.

- When multiple functions in a class with the same name but different parameters exist, these functions are said to be overloaded.
- The functions can be overloaded by using a different number of arguments and by using different types of arguments.
- If two same name and same argument functions just vary in their return type then such function isn't overloaded.

Function Overloading

```cpp
#include<iostream>
using namespace std;

class Student
{
public:

    void add(int a, int b)
    {
        cout << a + b << endl;
    }
    void add(int a, int b, int c)
    {
        cout << a + b + c << endl;
    }
    void add()
    {
        cout << "Default" << endl;
    }
};

int main()
{
    Student obj;
    obj.add(10, 20);
    obj.add(10, 20, 30);
    obj.add();
}

```

Function Overloading

```cpp
#include <bits/stdc++.h>
using namespace std;

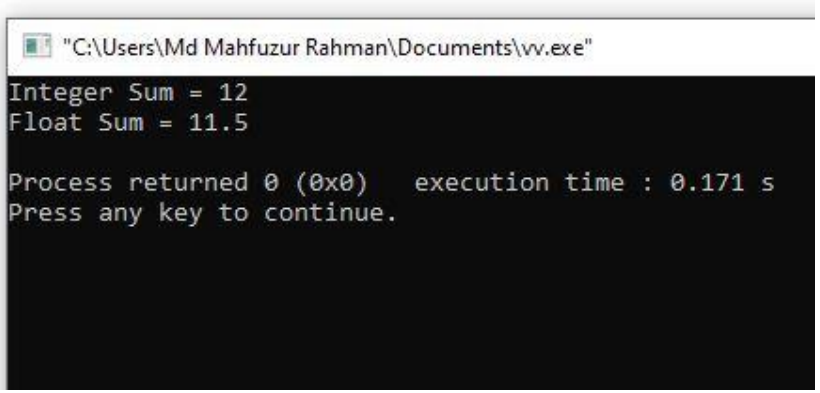
class Student {
public:

    void add(int a, int b) {
        cout << "Integer Sum = " << a + b << endl;
    }

    void add(double a, double b) {
        cout << "Float Sum = " << a + b << endl ;
    }
};

int main() {
    Student ob;

    ob.add(10, 2);
    ob.add(5.3, 6.2);
    return 0;
}
```



Explanation: In the above example, two **add()** functions are defined with the same name but different parameter types: one for integers and one for floating-point numbers. The correct version of the **add()** function is called at compile time based on the type of arguments passed, allowing the same function name to be used for different data types.

C++ Function Overloading Example

```
#include <iostream>
using namespace std;
class Cal {
    public:
static int add(int a, int b){
        return a + b;
    }
static int add(int a, int b, int c)
    {
        return a + b + c;
    }
};
```

```
int main(void) {
    Cal C;                  //      class object declaration.

    cout<<C.add(10, 20)<<endl;
    cout<<C.add(12, 20, 23);
   return 0;
}
```

Output:

30

55

Operator Overloading

We know that, "+" is used for addition or concatenation. Now, if I want to perform my customized operation i.e. upon calling +, I want to print "Hello Geek" or I want to perform subtraction instead of addition. In such cases, we use operator overloading.

Operator overloading is the process of defining different operations for the operator that vary depending on the argument type.

- Precedence and associativity remain intact in operators.
- List of operators that cannot be overloaded in C++ are **::, .*, ., ?:**
- Operators = and & are already overloaded in C++, so we should avoid overloading them.

Operator Overloading

Operator overloading is a compile-time polymorphism in which the operator is overloaded to provide the special meaning to the user-defined data type. Operator overloading is used to overload or redefines most of the operators available in C++. It is used to perform the operation on the user-defined data type.

For example, we can make use of the addition operator (+) for string class to concatenate two strings. We know that the task of this operator is to add two operands. So a single operator '+', when placed between integer operands, adds them and when placed between string operands, concatenates them.

Operator that cannot be overloaded are as follows:

- Scope operator (::)
- Sizeof
- member selector(.)
- member pointer selector(*)
- ternary operator(?:)

Syntax of Operator Overloading

```
return_type class_name  : : operator op(argument_list)
{
    // body of the function.
}
```

- Where the **return type** is the type of value returned by the function.
- **class_name** is the name of the class.
- **operator op** is an operator function where op is the operator being overloaded, and the operator is the keyword.

Rules for Operator Overloading

- Existing operators can only be overloaded, but the new operators cannot be overloaded.
- The overloaded operator contains at least one operand of the user-defined data type.
- We cannot use friend function to overload certain operators. However, the member function can be used to overload those operators.
- When unary operators are overloaded through a member function take no explicit arguments, but, if they are overloaded by a friend function, takes one argument.
- When binary operators are overloaded through a member function takes one explicit argument, and if they are overloaded through a friend function takes two explicit arguments.

Operator Overloading

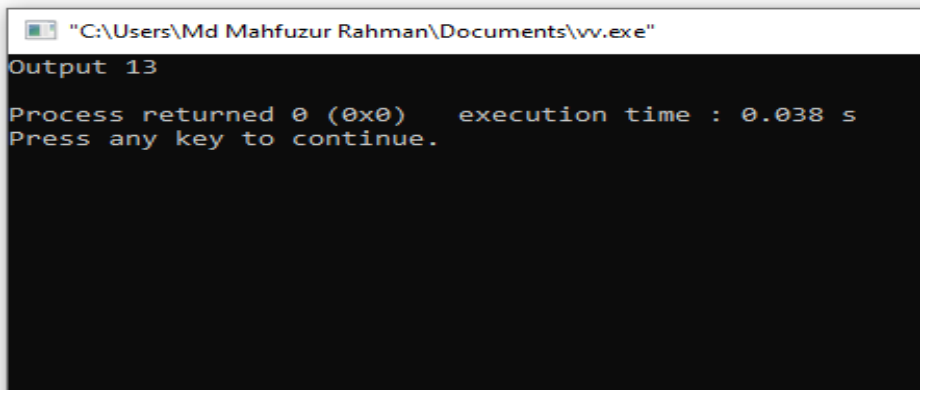
```cpp
#include <iostream>
using namespace std;

class B {
public:
    int a, b;

public:
    int add() { return a + b; }

    // overloading + operator for B class
    void operator+(B& obj)
    {
        int val1 = this->a;
        int val2 = obj.a;
        cout << "Output " << val2 - val1 << endl;
    }
};

// driver code
int main()
{
    B obj1, obj2;
    obj1.a = 4;
    obj2.a = 17;
    obj1 + obj2;
    return 0;
}
```



Operator Overloading

```cpp
#include <iostream>
using namespace std;

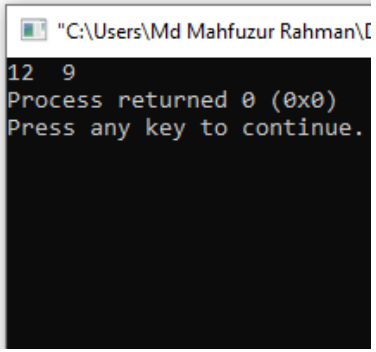
class Complex {
public:
    int real, imag;

    Complex(int r, int i) :
    real(r), imag(i) {}

    // Overloading the '+' operator
    Complex operator+(const Complex& obj) {
        return Complex(real + obj.real,imag + obj.imag);
    }
};

int main() {
    Complex c1(10, 5), c2(2, 4);

    // Adding c1 and c2 using + operator
    Complex c3 = c1 + c2;
    cout << c3.real << "  " << c3.imag;
    return 0;
}
```



C++ Operators Overloading Example

```
#include <iostream>
using namespace std;
class Test
{
   private:
      int num;
   public:
      Test(): num(8){}
      void operator ++()        {
         num = num+2;
      }
      void Print() {
         cout<<"The Count is: "<<num;
      }
};
```

```
int main()
{
    Test tt;
    ++tt;
    tt.Print();
    return 0;
}
```

Output:

The Count is: 10

```
#include <iostream>
using namespace std;
class A
{

    int x;
      public:
      A(){}
    A(int i)
    {
       x=i;
    }
    void operator+(A);
    void display();
};
```

```
void A :: operator+(A a)
{

    int m = x+a.x;
    cout<<"The result of the addition of two objects is : "<<m;
}
int main()
{
    A a1(5);
    A a2(4);
    a1+a2;
    return 0;
}
```

Output:

The result of the addition of two objects is : 9

2. Runtime Polymorphism

Also known as **late binding** and **dynamic polymorphism**, the function call in **runtime polymorphism** is resolved at runtime in contrast with compile time polymorphism, where the compiler determines which function call to bind at compilation. Runtime polymorphism is implemented using function overriding with virtual functions.

In run-time polymorphism, the decision of which function to call is determined at runtime based on the actual object type rather than the reference or pointer type. It is also known as Dynamic Polymorphism because the function calls are dynamically bonded at the runtime. Run Time Polymorphism can be exhibited by:

- Method Overriding using Virtual Functions

Function Overriding

Function Overriding occurs when a derived class defines one or more member functions of the base class. That base function is said to be **overridden**. The base class function must be declared as **virtual function** for runtime polymorphism to happen.

Method overriding refers to the process of creating a new definition of a function in a derived class that is already defined inside its base class. Some rules that must be followed while overriding a method are:

- Method names must be the same.
- Method parameters must be the same.

Virtual Function

- Virtual Function is a member function that is declared as virtual in the base class and it can be overridden in the derived classes that inherit the base class.
- Virtual functions are generally declared in the base class and are typically defined in both the base and derived classes.

Function Overriding

```cpp
#include<iostream>
using namespace std;

class Person
    {
    public:
        void display()
        {
            cout <<" I am a Person" << endl;
        }
    };
class Student : public Person
{
public:
    void display()
        {
            cout <<" I am a Student" << endl;
        }
};
class Teacher : public Person
{
public:
    void display()
        {
            cout <<" I am a Teacher" << endl;
        }
};
```

```cpp
int main()
{
    Person p;
    p.display();

    Student s;
    s.display();

    Teacher t;
    t.display();
}
```

C++ Function Overriding

If derived class defines same function as defined in its base class, it is known as function overriding in C++. It is used to achieve runtime polymorphism. It enables you to provide specific implementation of the function which is already provided by its base class.

```
#include <iostream>
using namespace std;
class Animal {
   public:
void eat(){
cout<<"Eating...";
   }
};
```

```
class Dog: public Animal
{
 public:
 void eat()
   {
      cout<<"Eating bread...";
   }
};
int main(void) {
   Dog d = Dog();
   d.eat();
   return 0;
}
```

Function Overloading & Overriding

```
class Overload
{
public :
    void add(int a,int b)
    {
        cout << a+b;
    }
    void add(int a,int b,int c)
    {
        cout << a+b+c;
    }
    void add()
    {
        cout << "Nothing to add";
    }

};
```

```
class Person
{
    public:
    void display()
    {
        cout << "I am a Person" <<endl;
    }
};
```

```
class Student : public Person
{
    public:
    void display()
    {
        cout << "I am a Student" <<endl;
    }
};
```

```
class Teacher : public Person
{
    public:
    void display()
    {
        cout << "I am a Teacher" <<endl;
    }
```

Function Overloading and Overriding

Method Overloading	Method Overriding
1. Parameter must be different.	1. Parameter must be different.
2. It occurs within the same class.	2. It occurs between two classes - sub class and a super class.
3. Inheritance is not involved.	3. Inheritance is involved.
4. Return type may or may not be same.	4. Return type must be same.
5. One method does not hide another.	5. child method hides parent another.

Compiletime Polymorphism

```cpp
#include <bits/stdc++.h>
using namespace std;

class Base {
public:
     void display() {
         cout << "Base class function";
     }
};

class Derived : public Base {
public:

     void display()  {
         cout << "Derived class function";
     }
};

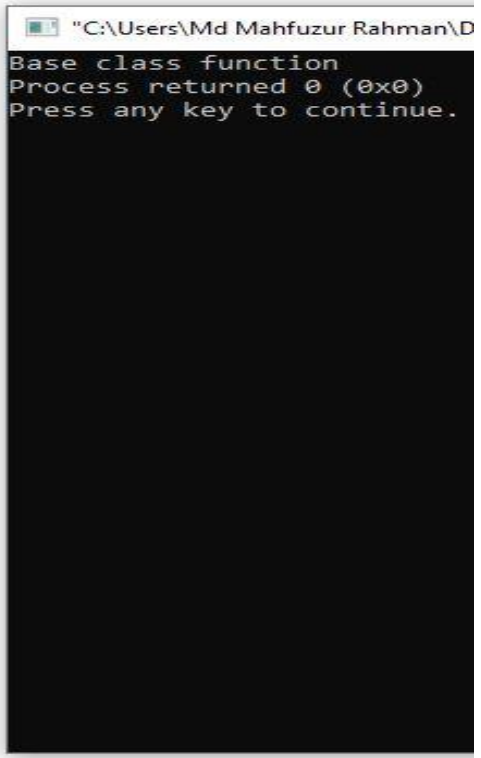
int main() {

     Base* basePtr;

     Derived derivedObj;

     basePtr = &derivedObj;

     basePtr->display();
     return 0;
}
```



Runtime Polymorphism

```cpp
#include <bits/stdc++.h>
using namespace std;

class Base {
public:
    virtual void display() {
        cout << "Base class function";
    }
};

class Derived : public Base {
public:

    void display() override {
        cout << "Derived class function";
    }
};

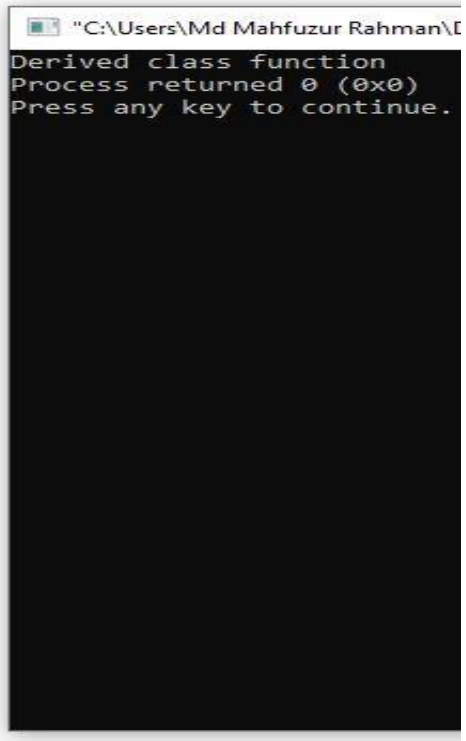
int main() {

    Base* basePtr;

    Derived derivedObj;

    basePtr = &derivedObj;

    basePtr->display();
    return 0;
}
```



Polymorphism (Dynamic binding)

```cpp
#include<iostream>
using namespace std;

class Person
    {
    public:
        virtual void display()
        {
            cout <<" I am a Person" << endl;
        }
    };
class Student : public Person
{
public:
    void display()
        {
            cout <<" I am a Student" << endl;
        }
};
class Teacher : public Person
{
public:
    void display()
        {
            cout <<" I am a Teacher" << endl;
        }
};
```

```cpp
int main()
{
    Student s;
    Teacher t;

    Person *p;
    p = &s;
    p->display();

    p = &t;
    p->display();
}
```

```
class Animal {
  public:
    void animalSound() {
      cout << "The animal makes a sound \n";
    }
};

class Pig : public Animal {
  public:
    void animalSound() {
      cout << "The pig says: wee wee \n";
    }
};

class Dog : public Animal {
  public:
    void animalSound() {
      cout << "The dog says: bow wow \n";
    }
};
```

Compile Time Polymorphism

```
int main() {
  Animal myAnimal;
  Pig myPig;
  Dog myDog;

  myAnimal.animalSound();
  myPig.animalSound();
  myDog.animalSound();
  return 0;
}
```

Run Time Polymorphism

Difference Between Compile Time And Run Time Polymorphism

Compile-Time Polymorphism	Run-Time Polymorphism
It is also called Static Polymorphism.	It is also known as Dynamic Polymorphism.
In compile-time polymorphism, the compiler determines which function or operation to call based on the number, types, and order of arguments.	In run-time polymorphism, the decision of which function to call is determined at runtime based on the actual object type rather than the reference or pointer type.
Function calls are statically binded.	Function calls are dynamically binded.
Compile-time Polymorphism can be exhibited by: 1. Function Overloading 2. Operator Overloading	Run-time Polymorphism can be exhibited by Function Overriding.
Faster execution rate.	Comparatively slower execution rate.
Inheritance in not involved.	Involves inheritance.

Runtime vs Compiler Time Polymorphism

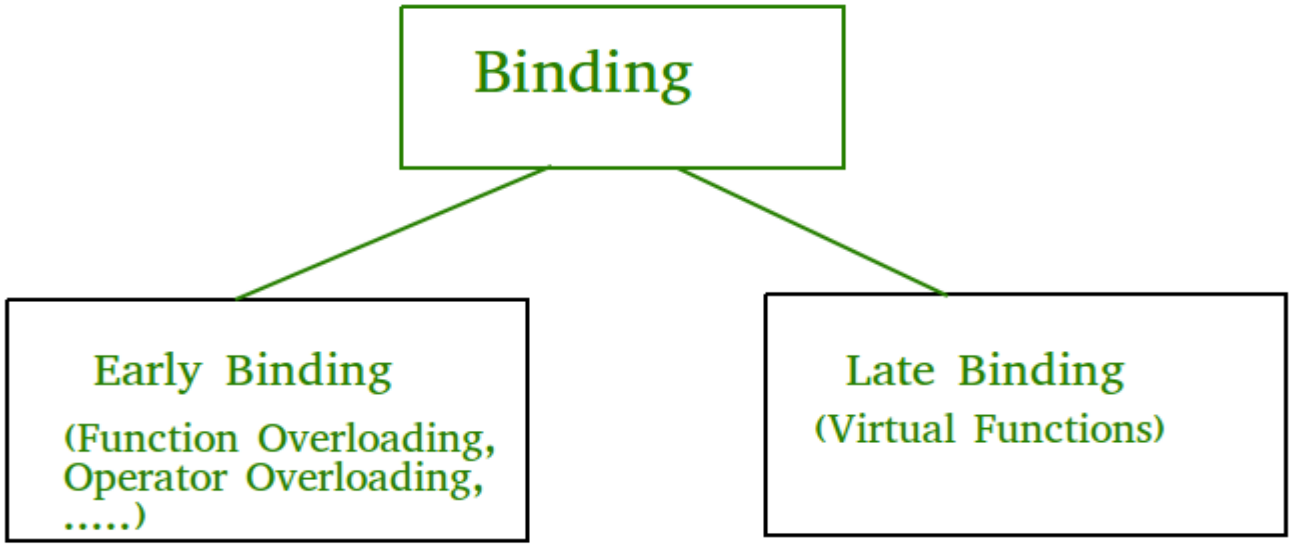
Compile Time Polymorphism	Run time Polymorphism
Also called static binding	Also called dynamic binding
Achieved using function overloading and operator overloading	Achieved using virtual functions and function overriding
Decision made by the compiler at compile time	Decision made at runtime using vtables
Faster due to early binding	More flexible but slightly slower

Differences between compile time and run time polymorphism.

Compile time polymorphism	Run time polymorphism
• The overloaded functions are invoked by matching the type and number of arguments. This information is available at the compile time and, therefore, compiler selects the appropriate function at the compile time.	• Run time polymorphism is achieved when the object's method is invoked at the run time instead of compile time.
• It is also known as overloading, early binding and static binding.	• It is also known as overriding, Dynamic binding and late binding.
• Overloading is a compile time polymorphism where more than one method is having the same name but with the different number of parameters or the type of the parameters.	• Overriding is a run time polymorphism where more than one method is having the same name, number of parameters and the type of the parameters.
• It is achieved by function overloading and operator overloading.	• It is achieved by virtual functions and pointers.
• It provides fast execution as it is known at the compile time.	• It provides slow execution as it is known at the run time.
• It is less flexible as mainly all the things execute at the compile time.	• It is more flexible as all the things execute at the run time.

Early binding and Late binding in C++

Binding refers to the process of converting identifiers (such as variable and performance names) into addresses. Binding is done for each variable and functions. For functions, it means that matching the call with the right function definition by the compiler. It takes place either at compile time or at runtime.



Early binding and Late binding in C++

Early Binding (compile-time time polymorphism) As the name indicates, compiler (or linker) directly associate an address to the function call. It replaces the call with a machine language instruction that tells the mainframe to leap to the address of the function. By default early binding happens in C++. Late binding (discussed below) is achieved with the help of virtual keyword)

Late Binding : (Run time polymorphism) In this, the compiler adds code that identifies the kind of object at runtime then matches the call with the right function definition (Refer this for details). This can be achieved by declaring a virtual function.

- **Early Binding:** It happens when a function call is resolved during the program's compilation. This makes it faster because everything is decided early
- **Late Binding:** It happens with virtual functions where the exact function to call is decided at runtime, depending on the actual object type. This is slower because the program has to figure it out while running.

Early binding

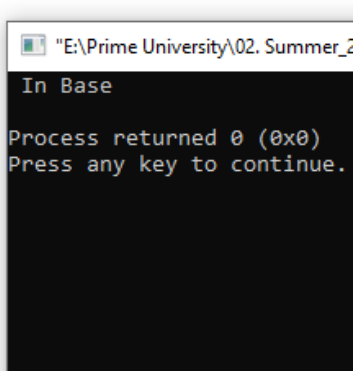
```cpp
#include<iostream>
using namespace std;

class Base
{
public:
    void show() { cout<<" In Base \n"; }
};

class Derived: public Base
{
public:
    void show() { cout<<"In Derived \n"; }
};

int main(void)
{
    Base *bp = new Derived;
    bp->show();

    return 0;
}
```



Early binding

```cpp
#include <iostream>
using namespace std;

class base {
public:
     void print() {
         cout << "print base class\n";
     }

     void show() {
         cout << "show base class\n";
     }
};

class derived : public base {
public:
     void print() {
         cout << "print derived class\n";
     }

     void show() {
         cout << "show derived class\n";
     }
};
```

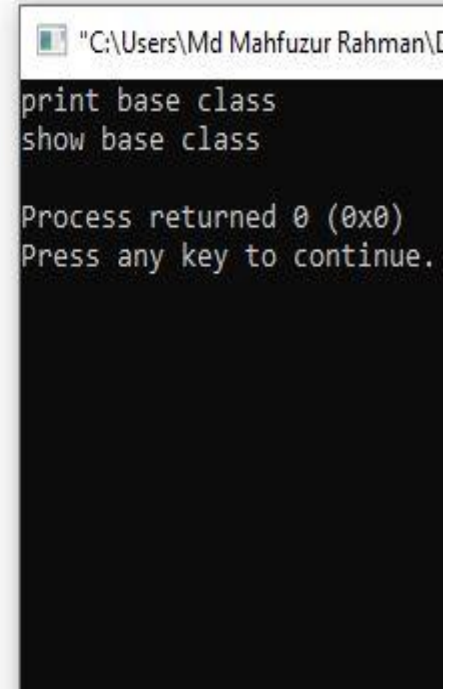
```cpp

int main() {
     base* bptr;
     derived d;
     bptr = &d;

     // Virtual function,
     // binded at runtime
     bptr->print();

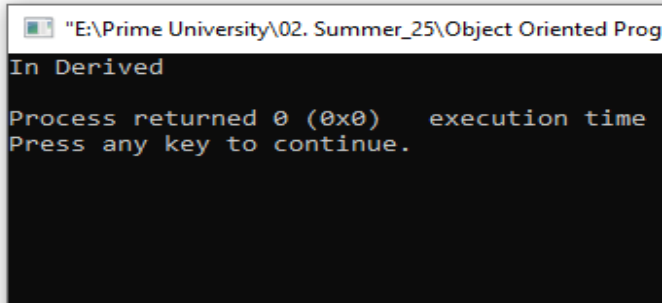
     // Non-virtual function,
     // binded at compile time
     bptr->show();
     return 0;
}

```



Dynamic binding

```
1   #include<iostream>
2   using namespace std;
3
4   class Base
5   {
6   public:
7       virtual void show() { cout<<" In Base \n"; }
8   };
9
10  class Derived: public Base
11  {
12  public:
13      void show() { cout<<"In Derived \n"; }
14  };
15
16  int main(void)
17  {
18      Base *bp = new Derived;
19      bp->show();
20      return 0;
21  }
22
```



Dynamic binding

```cpp
#include <iostream>
using namespace std;

class base {
public:
    virtual void print() {
        cout << "print base class\n";
    }

    void show() {
        cout << "show base class\n";
    }
};

class derived : public base {
public:
    void print() {
        cout << "print derived class\n";
    }

    void show() {
        cout << "show derived class\n";
    }
};
```

```cpp
int main() {
    base* bptr;
    derived d;
    bptr = &d;

    // Virtual function,
    // binded at runtime
    bptr->print();

    // Non-virtual function,
    // binded at compile time
    bptr->show();
    return 0;
}
```

```
"C:\Users\Md Mahfuzur Rahman\D
print derived class
show base class

Process returned 0 (0x0)
Press any key to continue.
```

C++ virtual function

- A C++ virtual function is a member function in the base class that you redefine in a derived class. It is declared using the virtual keyword.
- It is used to tell the compiler to perform dynamic linkage or late binding on the function.
- There is a necessity to use the single pointer to refer to all the objects of the different classes. So, we create the pointer to the base class that refers to all the derived objects. But, when base class pointer contains the address of the derived class object, always executes the base class function. This issue can only be resolved by using the 'virtual' function.
- A 'virtual' is a keyword preceding the normal declaration of a function.
- When the function is made virtual, C++ determines which function is to be invoked at the runtime based on the type of the object pointed by the base class pointer.

Rules for Virtual Functions

- The rules for the virtual functions in C++ are as follows:
- Virtual functions are defined in the base class and can be overridden in derived classes (not mandatory; base version is used if not overridden).
- They must have the same prototype in base and derived classes.
- They are used through a base class pointer or reference to achieve runtime polymorphism.
- A class may have a virtual destructor in case of dynamic memory allocation, but never a virtual constructor.
- Virtual functions cannot be static, but they can be friend functions of another class.

Rules of Virtual Function

- Virtual functions must be members of some class.
- Virtual functions cannot be static members.
- They are accessed through object pointers.
- They can be a friend of another class.
- A virtual function must be defined in the base class, even though it is not used.
- The prototypes of a virtual function of the base class and all the derived classes must be identical. If the two functions with the same name but different prototypes, C++ will consider them as the overloaded functions.
- We cannot have a virtual constructor, but we can have a virtual destructor
- Consider the situation when we don't use the virtual keyword.

```
#include <iostream>
{
 public:
 virtual void display()
 {
  cout << "Base class is invoked"<<endl;
 }
};
class B:public A
{
 public:
 void display()
 {
  cout << "Derived Class is invoked"<<endl;
 }
};
int main()
{
 A* a;    //pointer of base class
 B b;     //object of derived class
 a = &b;
 a->display();   //Late Binding occurs
}
```

Output:

Derived Class is invoked

Virtual Function in C++

A virtual function is a member function that is declared within a base class using the keyword **virtual** and is re-defined (Overridden) in the derived class. Virtual functions enable runtime polymorphism, calling the correct function via a base class pointer or reference.

- Virtual functions are mainly related to inheritance, allowing derived classes to provide their own implementation of base class functions.
- When a derived class object is deleted through a base class pointer, a virtual destructor in the base class ensures that both the derived and base class destructors are called, safely releasing all resources.
- The virtual destructor is declared using the keyword virtual in the base class.
- If a class has any virtual functions, its destructor should also be declared virtual. This is crucial for proper cleanup in polymorphic scenarios, especially when dealing with dynamic memory and inheritance.

Virtual Function

```cpp
#include <iostream>
using namespace std;

class Shape
{
  public:

    // Virtual function
    virtual void calculate()
    {
        cout << "Area of your Shape ";
    }

    // Virtual destructor
    virtual ~Shape()
    {
        cout << "Shape Destructor called\n";
    }
};

// Derived class: Rectangle
class Rectangle : public Shape
{
  public:
    int width, height, area;

    void calculate() override
    {
        width = 5;
        height = 10;

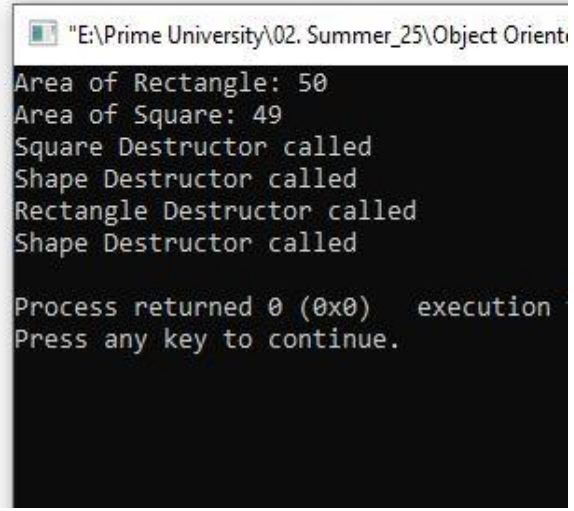
        area = height * width;
        cout << "Area of Rectangle: " << area << "\n";
    }

    ~Rectangle()
    {
        cout << "Rectangle Destructor called\n";
    }
};
```

Virtual Function

```
42  // Derived class: Square
43  class Square : public Shape
44  {
45    public:
46      int side, area;
47
48      void calculate() override
49      {
50          side = 7;
51          area = side * side;
52          cout << "Area of Square: " << area << "\n";
53      }
54
55      ~Square()
56      {
57          cout << "Square Destructor called\n";
58      }
59  };
```

```
60
61  int main()
62  {
63
64      Shape *S;
65
66      Rectangle r;
67      S = &r;
68      S->calculate();
69
70      Square sq;
71      S = &sq;
72      S->calculate();
73
74      return 0;
75  }
```



Pure Virtual Function

A **pure virtual function** is a function in a base class with = 0 and no body, which must be overridden in derived classes. A class with such a function is called abstract and cannot be instantiated.

A pure virtual function is a virtual function with no implementation in the base class, declared using = 0. A class with at least one pure virtual function is an abstract class that cannot be instantiated and serves as a blueprint for derived classes, which must provide their own implementation.

- A class with at least one pure virtual function becomes an abstract class and Objects of abstract classes cannot be created directly.
- Abstract classes are used to define interfaces and ensure common structure among derived classes.
- Useful in polymorphism where different classes share the same interface but have different behaviors.
- A pure virtual function forces derived classes to override it.
- virtual void draw() = 0; declares a pure virtual function, forcing derived classes to provide their own implementation.

Pure Virtual Function

- A virtual function is not used for performing any task. It only serves as a placeholder.
- When the function has no definition, such function is known as "**do-nothing**" function.
- The "**do-nothing**" function is known as a **pure virtual function**. A pure virtual function is a function declared in the base class that has no definition relative to the base class.
- A class containing the pure virtual function cannot be used to declare the objects of its own, such classes are known as abstract base classes.
- The main objective of the base class is to provide the traits to the derived classes and to create the base pointer used for achieving the runtime polymorphism.
- **Pure virtual function can be defined as:**

virtual void display() = 0;

```
#include <iostream>
using namespace std;
class Base
{
   public:
   virtual void show() = 0;
};
class Derived : public Base
{
   public:
   void show()
   {
      std::cout << "Derived class is derived from the base class." << std::endl;
   }
};
int main()
{
   Base *bptr;
   //Base b;
   Derived d;
   bptr = &d;
   bptr->show();
   return 0;
}
```

Output:

Derived class is derived from the base class.

In the above example, the base class contains the pure virtual function. Therefore, the base class is an abstract base class. We cannot create the object of the base class.

Pure Virtual Function

```cpp
#include <iostream>
using namespace std;

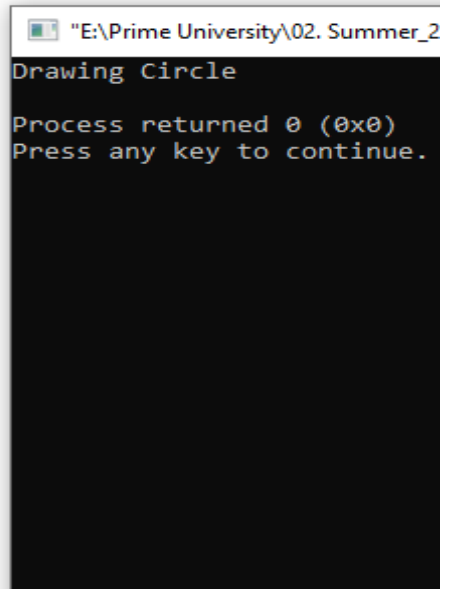
class Shape {
public:
    virtual void draw() = 0; // Pure virtual function
};

class Circle : public Shape {
public:
    void draw() override {
        cout << "Drawing Circle\n";
    }
};

int main() {

    Shape* s = new Circle();
    s->draw();

    delete s;
}
```



Pure Virtual Function

```cpp
#include <iostream>
using namespace std;
class Shape {
    public: virtual void draw() = 0;
};

class Circle: public Shape {
    public: void draw() override {
        cout << "Drawing a Circle" << endl;
    }
};

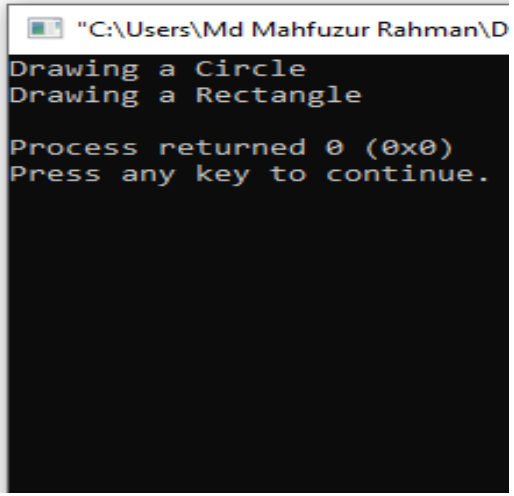
class Rectangle: public Shape {
    public: void draw() override {
        cout << "Drawing a Rectangle" << endl;
    }
};

int main() {
    Shape * shape1;
    Circle circleObj;
    Rectangle rectObj;

    shape1 = & circleObj;
    shape1 -> draw();

    shape1 = & rectObj;
    shape1 -> draw();

    return 0;
}
```



Pure Virtual Function

```cpp
#include <iostream>
using namespace std;

class Base
{
  public:

    // Pure virtual function
    virtual void display() = 0;

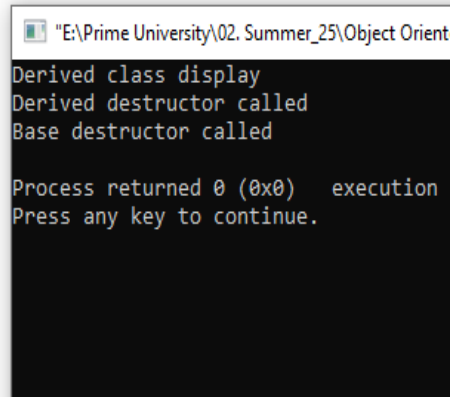
    // Pure virtual destructor
    virtual ~Base() = 0;
};

// Definition of pure virtual destructor
Base::~Base()
{
    cout << "Base destructor called" << endl;
}
```

```cpp
class Derived : public Base
{
  public:
    void display() override
    {
        cout << "Derived class display" << endl;
    }

    ~Derived()
    {
        cout << "Derived destructor called" << endl;
    }
};

int main()
{
    Base *basePtr;
    Derived derivedObj;
    basePtr = &derivedObj;
    basePtr->display();
    return 0;
}
```



Real-Life Example to Understand the Implementation of Virtual Function

Consider employee management software for an organization.

- We make a base class Employee with some common functions like raiseSalary() and promote(). These are declared virtual, so they can be overridden.
- Different types of employees (Manager, Engineer) inherit from Employee and override these functions with their own specific logic.
- In main(), we keep a list of Employee* pointers (pointing to both Manager and Engineer objects).
- When we loop through the list and call functions like raiseSalary() or promote(), polymorphism ensures that the correct version (Manager’s or Engineer’s) is called depending on the actual object type — even though we are using base class pointers.

Limitations of Virtual Functions

- **Slower:** The function call takes slightly longer due to the virtual mechanism and makes it more difficult for the compiler to optimize because it does not know exactly which function is going to be called at compile time.
- **Difficult to Debug:** In a complex system, virtual functions can make it a little more difficult to figure out where a function is being called from.

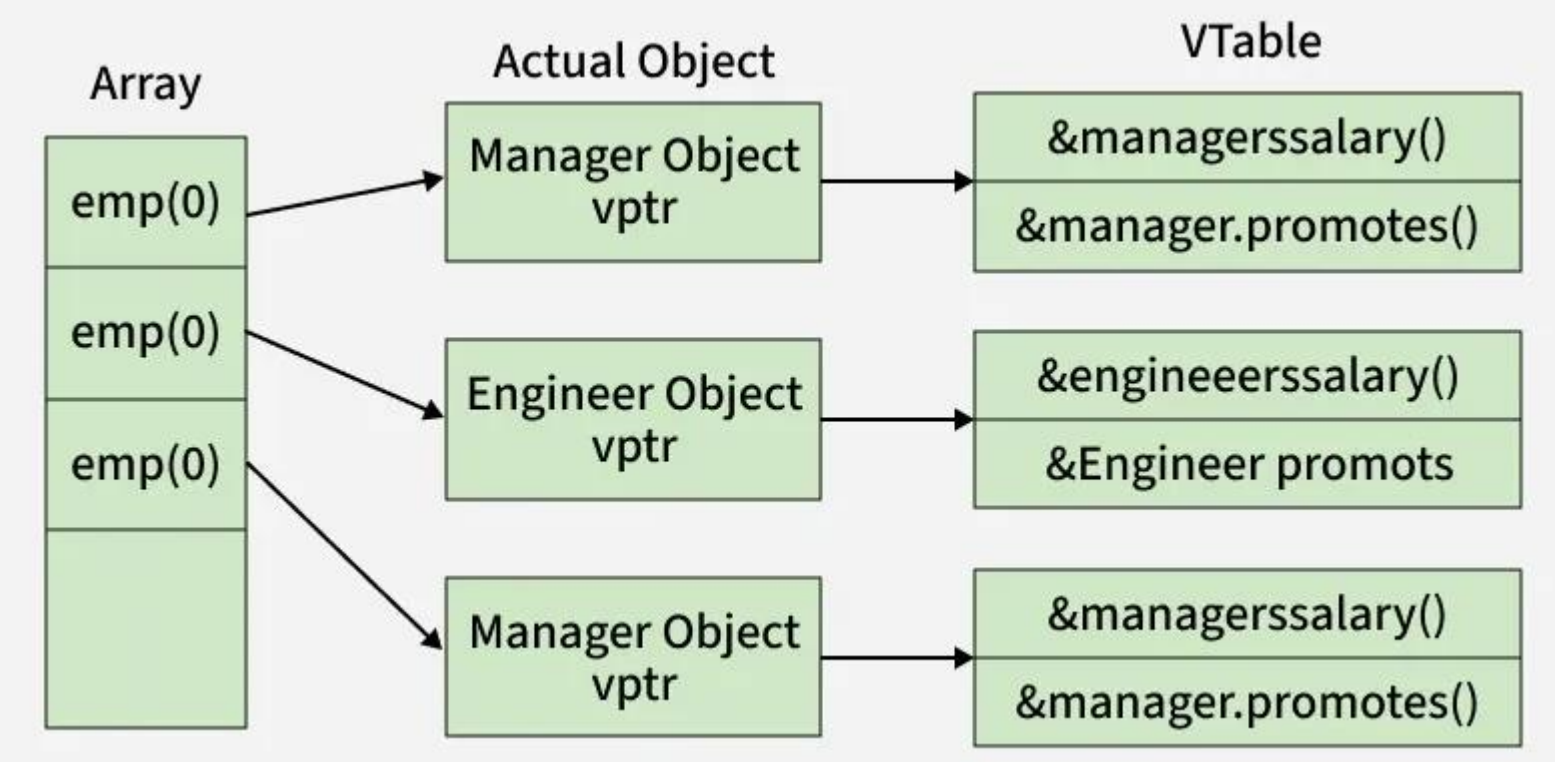
Pure virtual destructor

A **pure virtual destructor** is a destructor in a base class declared with = 0. It makes the class abstract and ensures that when you delete a derived class object through a base class pointer, the derived destructor runs first, followed by the base destructor, cleaning up everything properly.

How does the compiler perform runtime resolution?

The compiler maintains two things to serve this purpose:

- **vtable** : A table of function pointers, maintained per class.
- **vptr** : A pointer to vtable, maintained per object instance (see this for an example).



How does the compiler perform runtime resolution?

The compiler adds additional code at two places to maintain and use vptr.

- Code in every constructor. This code sets the vptr of the object being created. This code sets vptr to point to the vtable of the class.
- Code with polymorphic function call (e.g. bp->show() in above code). Wherever a polymorphic call is made, the compiler inserts code to first look for vptr using a base class pointer or reference (In the above example, since the pointed or referred object is of a derived type, vptr of a derived class is accessed). Once vptr is fetched, vtable of derived class can be accessed. Using vtable, the address of the derived class function show() is accessed and called.

What is Polymorphism?

- In OOPs, polymorphism refers to the ability of objects to have names but different functionality. The most common use is with base class pointers or references calling derived class functions. This is called runtime polymorphism and is implemented in C++ using virtual functions.

Virtual Functions in C++

- A virtual function is a member function that is declared within a base class with the virtual keyword and is re-defined (Overridden) by a derived class. When a class contains a virtual function, it can be overridden in its derived class and tells the compiler to perform dynamic linkage (or late binding) on the function. Now, the virtual functions are implemented in C++ by using vTable and vPtr.

What is vTable?

- The vTable, or Virtual Table, is a table of function pointers that is created by the compiler to support dynamic polymorphism. Whenever a class contains a virtual function, the compiler creates a Vtable for that class. Each object of the class is then provided with a hidden pointer to this table, known as Vptr.

- The Vtable has one entry for each virtual function accessible by the class. These entries are pointers to the **most derived function** that the current object should call.

What is vPtr (Virtual Pointer)?

- **The virtual pointer or _vptr** is a hidden pointer that is added by the compiler as a member of the class to point to the VTable of that class. For every object of a class containing virtual functions, a vptr is added to point to the vTable of that class. It's important to note that vptr is created only if a class has or inherits a virtual function.

vTable and vPtr

```cpp
#include <iostream>
using namespace std;

// base class
class Base {
public:
    virtual void function1()
    {
        cout << "Base function1()" << endl;
    }
    virtual void function2()
    {
        cout << "Base function2()" << endl;
    }
    virtual void function3()
    {
        cout << "Base function3()" << endl;
    }
};

// class derived from Base
class Derived1 : public Base {
public:
    // overriding function1()
    void function1()
    {
        cout << "Derived1 function1()" << endl;
    }
    // not overriding function2() and function3()
};
```

vTable and vPtr

```
32  // class derived from Derived1
33  class Derived2 : public Derived1 {
34  public:
35      // again overriding function2()
36      void function2()
37      {
38          cout << "Derived2 function2()" << endl;
39      }
40      // not overriding function1() and function3()
41  };
42
43  // driver code
44  int main()
45  {
46      // defining base class pointers
47      Base* ptr1 = new Base();
48      Base* ptr2 = new Derived1();
49      Base* ptr3 = new Derived2();
50
51      // calling all functions
52      ptr1->function1();
53      ptr1->function2();
54      ptr1->function3();
55      ptr2->function1();
56      ptr2->function2();
57      ptr2->function3();
58      ptr3->function1();
59      ptr3->function2();
60      ptr3->function3();
61
62      // deleting objects
63      delete ptr1;
64      delete ptr2;
65      delete ptr3;
66
67      return 0;
68  }
69
```

```
"E:\Prime University\02. Summer_25\Obje
Base function1()
Base function2()
Base function3()
Derived1 function1()
Base function2()
Base function3()
Derived1 function1()
Derived2 function2()
Base function3()

Process returned 0 (0x0)   exec
Press any key to continue.
```

Thank You